Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

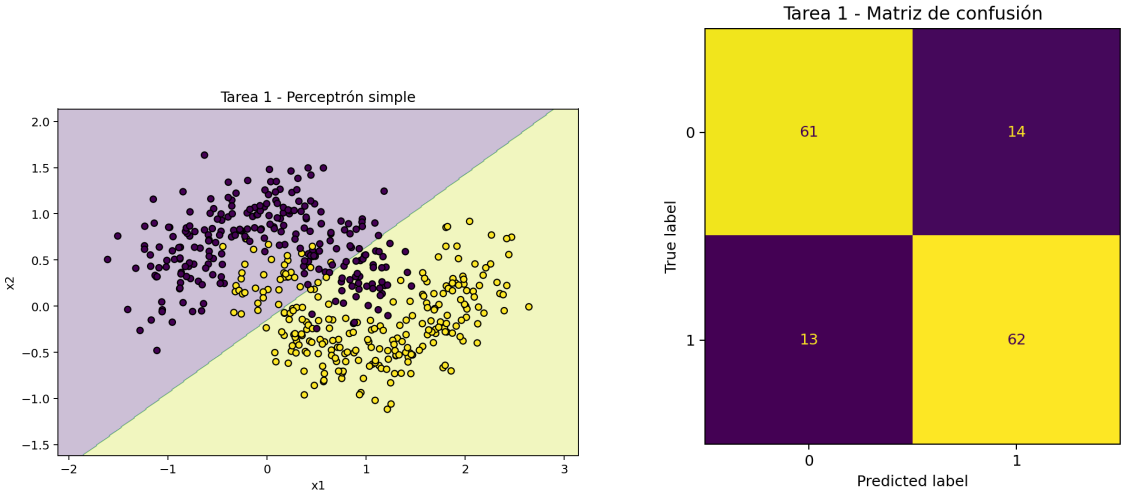
Arquitectura de Redes Neuronales: El Perceptrón Multicapa

En esta práctica se explora el funcionamiento del Perceptrón Multicapa (MLP) y su capacidad para modelar fronteras de decisión no lineales. A diferencia del perceptrón simple, el MLP aborda problemas de clasificación más complejos mediante la introducción de capas ocultas. A lo largo del documento se analizan distintos hiperparámetros, como el número de neuronas y la función de activación, y su impacto en la convergencia y generalización del modelo.

1.1. Tarea 1: La barrera de la linealidad — El Perceptrón Simple

Para iniciar el estudio, se ha evaluado el rendimiento de un Perceptrón simple (un modelo sin capas ocultas, definido con `hidden_layer_sizes=()`) aplicado al dataset sintético `make_moons`. Aunque tras el entrenamiento se alcanza una precisión en test de aproximadamente un 82 %, el análisis visual de las gráficas revela que el modelo falla de manera sistemática en ciertas regiones complejas.

1.1.1. Resultados y Análisis Estructural



(a) Frontera de decisión lineal.

(b) Matriz de confusión.

Figura 1.1: Evaluación del rendimiento en un perceptrón de una sola capa.

Al observar la frontera de decisión (Figura 1.1a), la causa del error se hace evidente. El dataset presenta dos clases distribuidas en forma de lunas entrelazadas, pero el modelo ha trazado un límite de separación completamente recto. Matemáticamente, es imposible separar estas dos clases con un solo hiperplano, lo que explica la distribución de fallos en la matriz de confusión (Figura 1.1b).

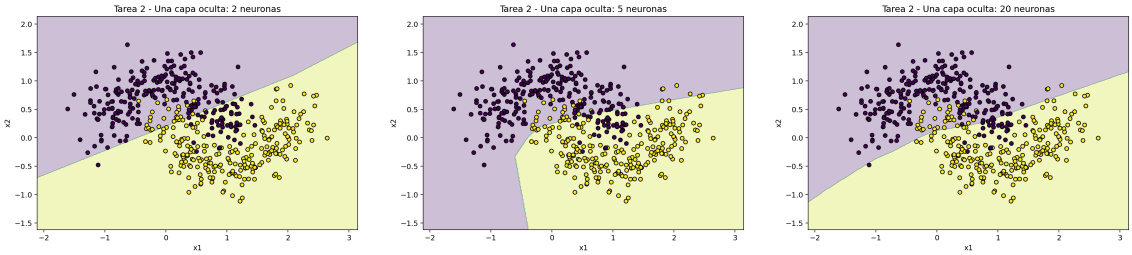
Este comportamiento no indica una falta de entrenamiento, sino una limitación inherente a la arquitectura. Al no disponer de capas ocultas, un perceptrón simple solo puede generar combinaciones lineales de las entradas, restringiendo su capacidad a la creación de fronteras lineales.

Se concluye que el modelo padece un grado severo de *underfitting* debido a su simplicidad. Para clasificar correctamente distribuciones que no son linealmente separables, es necesario dotar a la red de representaciones no lineales mediante el uso de capas ocultas.

1.2. Tarea 2: Impacto de la capacidad en la capa oculta

Una vez constatada la limitación del modelo lineal, se procede a introducir una capa oculta en el MLP para proporcionarle capacidad de decisión no lineal. Se han

evaluado tres configuraciones distintas (2, 5 y 20 neuronas) para ilustrar visualmente la adaptación de la frontera de decisión a medida que aumenta la complejidad de la red.



(a) 2 neuronas: Underfitting. (b) 5 neuronas: Transición. (c) 20 neuronas: Ajuste óptimo.

Figura 1.2: Evolución de la frontera de decisión según la densidad de la red.

Tabla 1.1: Resultados experimentales según la arquitectura.

Arquitectura	**Acc. Test**	**Iteraciones**	**Observación**
MLP(2)	0.8200	648	Capacidad insuficiente
MLP(5)	0.8400	726	Mayor esfuerzo de optimización
MLP(20)	0.8533	406	Convergencia eficiente

Configuración con 2 neuronas

Con dos neuronas, la red adquiere una ligera flexibilidad respecto al límite estático de la tarea anterior. No obstante, al observar la gráfica se aprecia que la frontera forma un ángulo sencillo en forma de pico. La capa carece de los parámetros suficientes para contornear la distribución, manteniendo la red un problema general de *underfitting*.

Configuración con 5 neuronas

Al escalar a 5 neuronas, el MLP consigue envolver ambas clases con mucha mayor efectividad. Resulta evidente en la representación que la frontera no es suavemente curva: al emplear la función de activación ReLU, la forma se delimita ensamblando segmentos de recta por cada neurona oculta. Este comportamiento resuelve las carencias iniciales, dando lugar a un contorno poligonal característico de redes con pocos nodos y activadas por ReLU.

Configuración con 20 neuronas

Finalmente, con 20 neuronas, la convergencia visual es notable, logrando definir una frontera de decisión mucho más precisa que se adapta al contorno de ambas clases sin ser excesivamente irregular. A pesar de incrementar la capacidad de la red, el modelo no incurre en *overfitting*, ya que evita adaptarse al ruido estadístico de los datos. Esto se confirma al observar que las métricas de *train* (86.00 %) y *test* (85.33 %) son prácticamente idénticas, mostrando una buena capacidad de generalización, aunque la mejora respecto a la configuración de 5 neuronas es marginal.

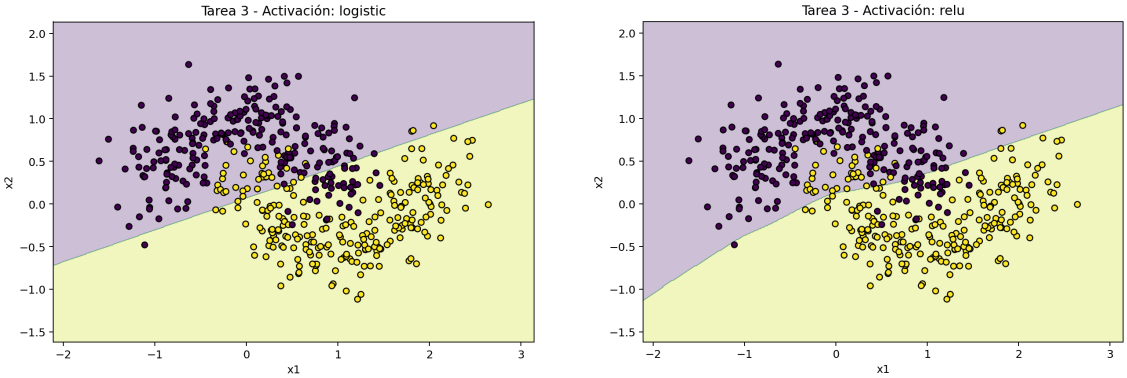
Conclusión

Se confirma visualmente que añadir neuronas a la capa oculta permite superar la limitación del perceptrón simple, dotando al modelo de la capacidad para trazar fronteras de decisión complejas (en este caso, aproximando curvas mediante la superposición de segmentos rectos generados por la activación ReLU). Este comportamiento ilustra el compromiso fundamental en el diseño de arquitecturas: la red debe tener suficientes neuronas para evitar el *underfitting*, pero sin sobredimensionarse, para no penalizar el coste computacional a cambio de mejoras insignificantes en la precisión.

1.3. Tarea 3: Dinámica de Aprendizaje — ReLU vs. Sigmoide

Manteniendo una arquitectura uniforme basada en una capa oculta de 20 neuronas, se expone ahora la comparativa entre la función logística (*Sigmoide*) y la unidad lineal rectificada (*ReLU*). El fin de este ejercicio es medir la eficacia en la separación y los tiempos de entrenamiento modificando únicamente el selector de activación de la red.

1.3.1. Fronteras de decisión



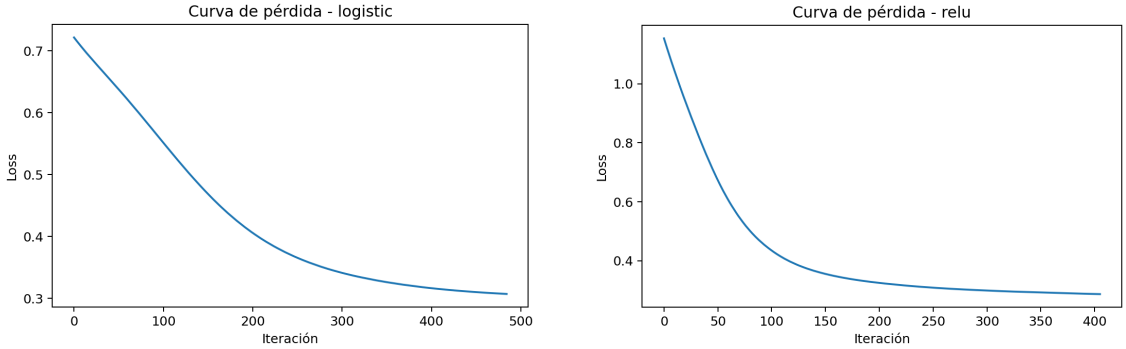
(a) Frontera con Sigmoide.

(b) Frontera con ReLU.

Figura 1.3: Comparación visual de la frontera de decisión según la activación.

Ambas funciones resuelven de forma adecuada la separación no lineal, aunque exhiben trazados característicos acordes a sus formulaciones algebraicas. La función Sigmoide genera una frontera de decisión suave y continua debido a su naturaleza asintótica en el intervalo (0, 1). Por el contrario, la activación ReLU produce un límite definido por la intersección de segmentos rectos, generando contornos más abruptos y poligonales.

1.3.2. Curvas de pérdida y velocidad de convergencia



(a) Curva de pérdida con Sigmoide.

(b) Curva de pérdida con ReLU.

Figura 1.4: Comparativa de la velocidad de aprendizaje.

Tabla 1.2: Métricas comparativas por función de activación (arquitectura `(20,)`).

Activación	Acc. Train	Acc. Test	Iteraciones	Loss final
Sigmoide (logistic)	0.8657	0.8400	485	0.3071
ReLU	0.8600	0.8533	406	0.2870

Análisis

El comportamiento visualizado en la Figura 1.4 se corrobora con las métricas (Tabla 1.2). Al finalizar el entrenamiento, el modelo con ReLU converge en 406 iteraciones, aventajando claramente a su homólogo con función Sigmoide, que necesita 485 iteraciones para alcanzar el mismo criterio de parada.

Esta diferencia de rendimiento se debe al problema del desvanecimiento del gradiente (*vanishing gradient*). La función Sigmoide satura sus salidas en los extremos (0 y 1), lo que provoca que su derivada tienda a cero. Durante el *backpropagation*, al multiplicar estos gradientes casi nulos, la actualización de los pesos en las primeras capas se paraliza, ralentizando la convergencia. Por el contrario, la función ReLU tiene una derivada constante de 1 para cualquier valor positivo. Esto permite que el error fluya hacia atrás sin atenuarse, evitando la congelación de los coeficientes y logrando una convergencia mucho más rápida (406 iteraciones frente a 485).

Aunque en este problema bidimensional sencillo ambas funciones logran resultados aceptables (entorno al 84-85 %), en arquitecturas multicapa más profundas la saturación de la Sigmoide provoca caídas críticas de rendimiento, siendo esta la razón por la que ReLU es el estándar actual.

1.4. Tarea 4: Reconocimiento de Dígitos y Simplicidad de la Arquitectura

Como prueba final de diseño de un MLP, el objetivo es alcanzar una precisión superior al 95 % en la clasificación del dataset de dígitos escritos a mano (`load_digits`), manteniendo la arquitectura de la red lo más simple posible. Para ello, se han evaluado seis configuraciones diferentes, variando la profundidad (número de capas) y la anchura (neuronas por capa), todas ellas entrenadas utilizando el optimizador Adam.

Tabla 1.3: Métricas obtenidas para distintas arquitecturas en el dataset de dígitos.

Arquitectura	Acc. Train	Acc. Test	Iteraciones	Profundidad	Meta
MLP(20)	1.0000	0.9644	276	1 capa	✓
MLP(50)	1.0000	0.9711	180	1 capa	✓
MLP(100)	1.0000	0.9800	147	1 capa	✓
MLP(30, 15)	1.0000	0.9600	161	2 capas	✓
MLP(50, 20)	1.0000	0.9756	131	2 capas	✓
MLP(64, 32)	1.0000	0.9667	110	2 capas	✓

1.4.1. Análisis de sobreajuste

Al analizar la Tabla 1.3, se observa que todas las arquitecturas alcanzan un 100 % de precisión en entrenamiento (`acc_train` = 1.0000). Esta asimetría frente a los resultados de test indica un claro sobreajuste (*overfitting*) inicial. Es un comportamiento previsible: estamos aplicando redes con alta capacidad de representación sobre un conjunto de datos relativamente pequeño (aprox. 1.350 muestras de validación cruzada).

A pesar de memorizar el conjunto de entrenamiento, el modelo no pierde su capacidad de generalización (manteniendo un 96-98 % en test). Esto se logra gracias al término de regularización L2 implementado por defecto en Scikit-Learn (`alpha=1e-4`), que penaliza el crecimiento descontrolado de los pesos y evita que la red se ajuste al ruido estadístico.

Cabe destacar el papel crítico de este hiperparámetro en el compromiso entre capacidad y generalización. Si eliminásemos esta regularización (`alpha=0`), la frontera de decisión se volvería excesivamente irregular, adaptándose a cada punto de ruido y desplomando la precisión en test. Por el contrario, si aplicásemos una penalización demasiado estricta (ej. `alpha=10`), forzaríamos los pesos a valores cercanos a cero. Esto limitaría artificialmente la capacidad expresiva de la red y provocaría un *underfitting* severo, devolviéndonos a un escenario de fallo similar al observado con el perceptrón simple en la Tarea 1.

1.4.2. Elección del modelo

Al evaluar la topología a partir de los datos de la Tabla 1.3, surge el debate clásico: ¿es preferible ensanchar una sola capa o añadir profundidad con una segunda capa? En problemas complejos de visión artificial, extraer características jerárquicas con redes profundas es vital. Sin embargo, para este dataset concreto (`load_digits`, imágenes de 8x8 píxeles), añadir una segunda capa oculta (como en MLP(30, 15) o

MLP(64, 32)) no aporta una ventaja competitiva real frente a ensanchar una sola. De hecho, las topologías de dos capas logran precisiones similares (entorno al 96-97 %) pero complican la arquitectura de forma innecesaria. Para este nivel de complejidad, la experimentación demuestra que una sola capa ancha es suficiente para capturar las no linealidades sin dificultar el entrenamiento.

Aunque el modelo MLP(100) alcanza la precisión más alta (98.00 %), bajo un criterio de ingeniería eficiente la opción óptima es el modelo MLP(20). Esta arquitectura más simple supera holgadamente el requisito del 95 % de precisión planteado en la práctica, pero siendo mucho más ligera computacionalmente. Para ponerlo en perspectiva, el clasificador MLP(20) solo necesita optimizar alrededor de 1.500 parámetros frente a los más de 7.500 que exige la arquitectura de 100 neuronas, logrando una eficacia prácticamente idéntica con una quinta parte del coste estructural.

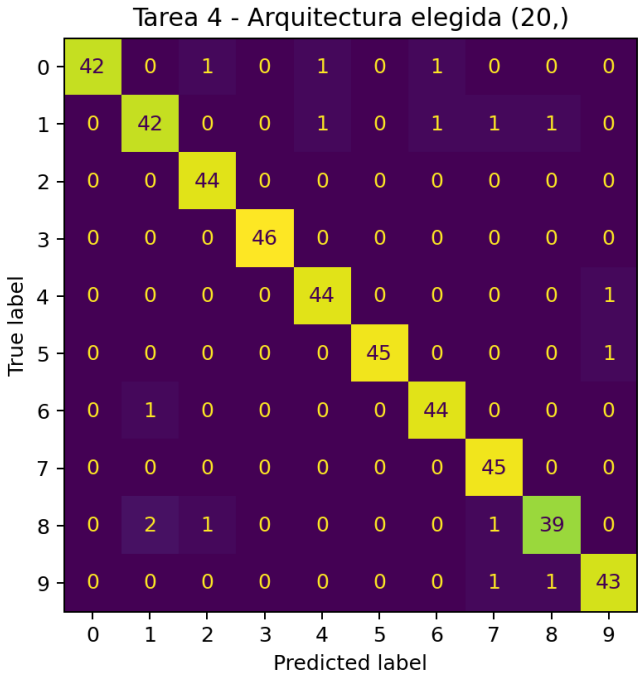


Figura 1.5: Matriz de confusión de la arquitectura **MLP(20)**. El desempeño es casi impecable en todas las clases; los escasos errores se concentran en pares de dígitos visualmente similares.

1.5. Propuesta Técnica de Despliegue

Finalizada la validación experimental, se exponen las decisiones de diseño recomendadas para la integración del modelo clasificador en un entorno de producción:

1. Arquitectura recomendada La configuración idónea es el modelo **MLP(20)**, compuesto por una única capa oculta de 20 neuronas y activado mediante la función **ReLU**. Se descarta el uso de un modelo lineal (cero capas ocultas) porque es estructuralmente incapaz de resolver fronteras de decisión complejas. La función ReLU se establece como el estándar preferido frente a la Sigmoide, ya que mitiga el desvanecimiento del gradiente y logra una convergencia más estable. Con esta configuración, el sistema alcanza un 96.44 % de precisión, superando el objetivo técnico del 95 %.

2. Justificación de simplicidad vs. profundidad Aunque el incremento a 100 neuronas mejora la precisión ligeramente (hasta un ∼98 %), supone multiplicar por cinco la cantidad de parámetros de la red. En un entorno real, esta ganancia marginal no compensa el sobrecoste computacional. Mantener la topología en 20 neuronas proporciona latencias más bajas y un menor consumo de memoria, facilitando su posible ejecución en arquitecturas de recursos limitados (*Edge Computing*). Además, al limitar la capacidad de la red, se reduce el riesgo de *overfitting*, haciendo que el modelo sea más robusto frente a variaciones en los datos de producción (*Data Drift*).

3. Estrategia de monitorización Debido a la tendencia inicial de la red a memorizar el conjunto de entrenamiento, se recomienda desplegar el modelo inicialmente bajo la modalidad de *shadow deployment* (despliegue en sombra). Esto permitirá evaluar la fiabilidad de sus predicciones frente al tráfico de datos real en tiempo de ejecución, pero sin impactar los procesos operativos hasta confirmar su total estabilización.